`urllib.request` — Extensible library for opening URLs

Source code: Lib/urllib/request.py

The `urllib.request` module defines functions and classes which help in opening URLs (mostly HTTP) in a complex world — basic and digest authentication, redirections, cookies and more.

See also: The Requests package is recommended for a higher-level HTTP client interface.

Warning: On macOS it is unsafe to use this module in programs using `os.fork()` because the `getproxies()` implementation for macOS uses a higher-level system API. Set the environment variable `no_proxy` to `*` to avoid this problem (e.g. `os.environ["no_proxy"] = "*"`).

Availability: not WASI.

This module does not work or is not available on WebAssembly. See WebAssembly platforms for more information.

The `urllib.request` module defines the following functions:

`urllib.request.`**`urlopen`**`(`*`url`*`, `*`data=None`*`, [`*`timeout`*`, ]*, `*`context=None`*`)`

Open *url*, which can be either a string containing a valid, properly encoded URL, or a `Request` object.

data must be an object specifying additional data to be sent to the server, or `None` if no such data is needed. See `Request` for details.

urllib.request module uses HTTP/1.1 and includes `Connection:close` header in its HTTP requests.

The optional *timeout* parameter specifies a timeout in seconds for blocking operations like the connection attempt (if not specified, the global default timeout setting will be used). This actually only works for HTTP, HTTPS and FTP connections.

If *context* is specified, it must be a `ssl.SSLContext` instance describing the various SSL options. See `HTTPSConnection` for more details.

This function always returns an object which can work as a context manager and has the properties *url*, *headers*, and *status*. See `urllib.response.addinfourl` for more detail on these properties.

For HTTP and HTTPS URLs, this function returns a `http.client.HTTPResponse` object slightly modified. In addition to the three new methods above, the msg attribute contains the same information as the `reason` attribute — the reason phrase returned by server — instead of the response headers as it is specified in the documentation for `HTTPResponse`.

For FTP, file, and data URLs, this function returns a `urllib.response.addinfourl` object.

Raises `URLError` on protocol errors.

Note that `None` may be returned if no handler handles the request (though the default installed global `OpenerDirector` uses `UnknownHandler` to ensure this never happens).

In addition, if proxy settings are detected (for example, when a `*_proxy` environment variable like `http_proxy` is set), `ProxyHandler` is default installed and makes sure the requests are handled through the proxy.

The legacy `urllib.urlopen` function from Python 2.6 and earlier has been discontinued; `urllib.request.urlopen()` corresponds to the old `urllib2.urlopen`. Proxy handling, which was done by passing a dictionary parameter to `urllib.urlopen`, can be obtained by using `ProxyHandler` objects.

The default opener raises an auditing event `urllib.Request` with arguments `fullurl`, `data`, `headers`, `method` taken from the request object.

> *Changed in version 3.2:* *cafile* and *capath* were added.
>
> HTTPS virtual hosts are now supported if possible (that is, if `ssl.HAS_SNI` is true).
>
> *data* can be an iterable object.

> *Changed in version 3.3:* *cadefault* was added.

> *Changed in version 3.4.3:* *context* was added.

> *Changed in version 3.10:* HTTPS connection now send an ALPN extension with protocol indicator `http/1.1` when no *context* is given. Custom *context* should set ALPN protocols with `set_alpn_protocols()`.

> *Changed in version 3.13:* Remove *cafile*, *capath* and *cadefault* parameters: use the *context* parameter instead.

`urllib.request.`**`install_opener`**`(`*`opener`*`)`

Install an `OpenerDirector` instance as the default global opener. Installing an opener is only necessary if you want urlopen to use that opener; otherwise, simply call `OpenerDirector.open()` instead of `urlopen()`. The code does not check for a real `OpenerDirector`, and any class with the appropriate interface will work.

`urllib.request.`**`build_opener`**`(`[*`handler`*`, ...`]`)`

Return an `OpenerDirector` instance, which chains the handlers in the order given. *handler*s can be either instances of `BaseHandler`, or subclasses of `BaseHandler` (in which case it must be possible to call the constructor without any parameters). Instances of the following classes will be in front of the *handler*s, unless the *handler*s contain them, instances of them or subclasses of them: `ProxyHandler` (if proxy settings are detected), `UnknownHandler`, `HTTPHandler`, `HTTPDefaultErrorHandler`, `HTTPRedirectHandler`, `FTPHandler`, `FileHandler`, `HTTPErrorProcessor`.

If the Python installation has SSL support (i.e., if the `ssl` module can be imported), `HTTPSHandler` will also be added.

A `BaseHandler` subclass may also change its `handler_order` attribute to modify its position in the handlers list.

`urllib.request.`**`pathname2url`**`(`*`path`*`, *, `*`add_scheme=False`*`)`

Convert the given local path to a `file:` URL. This function uses `quote()` function to encode the path.

If *add_scheme* is false (the default), the return value omits the `file:` scheme prefix. Set *add_scheme* to true to return a complete URL.

This example shows the function being used on Windows:

```
>>> from urllib.request import pathname2url
>>> path = 'C:\\Program Files'
>>> pathname2url(path, add_scheme=True)
'file:///C:/Program%20Files'
```

> *Changed in version 3.14:* Windows drive letters are no longer converted to uppercase, and `:` characters not following a drive letter no longer cause an `OSError` exception to be raised on Windows.

> *Changed in version 3.14:* Paths beginning with a slash are converted to URLs with authority sections. For example, the path `/etc/hosts` is converted to the URL `///etc/hosts`.

> *Changed in version 3.14:* The *add_scheme* parameter was added.

`urllib.request.`**`url2pathname`**`(`*`url`*`, *, `*`require_scheme=False`*`, `*`resolve_host=False`*`)`

Convert the given `file:` URL to a local path. This function uses `unquote()` to decode the URL.

If *require_scheme* is false (the default), the given value should omit a `file:` scheme prefix. If *require_scheme* is set to true, the given value should include the prefix; a `URLError` is raised if it doesn't.

The URL authority is discarded if it is empty, `localhost`, or the local hostname. Otherwise, if *resolve_host* is set to true, the authority is resolved using `socket.gethostbyname()` and discarded if it matches a local IP address (as per **RFC 8089 §3**). If the authority is still unhandled, then on Windows a UNC path is returned, and on other platforms a `URLError` is raised.

This example shows the function being used on Windows:

```
>>> from urllib.request import url2pathname
>>> url = 'file:///C:/Program%20Files'
>>> url2pathname(url, require_scheme=True)
'C:\\Program Files'
```

> *Changed in version 3.14:* Windows drive letters are no longer converted to uppercase, and `:` characters not following a drive letter no longer cause an `OSError` exception to be raised on Windows.

> *Changed in version 3.14:* The URL authority is discarded if it matches the local hostname. Otherwise, if the authority isn't empty or `localhost`, then on Windows a UNC path is returned (as before), and on other platforms a `URLError` is raised.

> *Changed in version 3.14:* The URL query and fragment components are discarded if present.

> *Changed in version 3.14:* The *require_scheme* and *resolve_host* parameters were added.

`urllib.request.`**`getproxies`**`()`

This helper function returns a dictionary of scheme to proxy server URL mappings. It scans the environment for variables named `<scheme>_proxy`, in a case insensitive approach, for all operating systems first, and when it cannot find it, looks for proxy information from System Configuration for macOS and Windows Systems Registry for Windows. If both lowercase and uppercase environment variables exist (and disagree), lowercase is preferred.

> **Note:** If the environment variable `REQUEST_METHOD` is set, which usually indicates your script is running in a CGI environment, the environment variable `HTTP_PROXY` (uppercase `_PROXY`) will be ignored. This is because that variable can be injected by a client using the "Proxy:" HTTP header. If you need to use an HTTP proxy in a CGI environment, either use `ProxyHandler` explicitly, or make sure the variable name is in lowercase (or at least the `_proxy` suffix).

The following classes are provided:

class `urllib.request.`**`Request`**`(`*`url, data=None, headers={}, origin_req_host=None, unverifiable=False, method=None`*`)`

This class is an abstraction of a URL request.

url should be a string containing a valid, properly encoded URL.

data must be an object specifying additional data to send to the server, or `None` if no such data is needed. Currently HTTP requests are the only ones that use *data*. The supported object types include bytes, file-like objects, and iterables of bytes-like objects. If no `Content-Length` nor `Transfer-Encoding` header field has been provided, `HTTPHandler` will set these headers according to the type of *data*. `Content-Length` will be used to send bytes objects, while `Transfer-Encoding: chunked` as specified in **RFC 7230**, Section 3.3.1 will be used to send files and other iterables.

For an HTTP POST request method, *data* should be a buffer in the standard *application/x-www-form-urlencoded* format. The `urllib.parse.urlencode()` function takes a mapping or sequence of 2-tuples and returns an ASCII string in this format. It should be encoded to bytes before being used as the *data* parameter.

headers should be a dictionary, and will be treated as if `add_header()` was called with each key and value as arguments. This is often used to "spoof" the `User-Agent` header value, which is used by a browser to identify itself – some HTTP servers only allow requests coming from common browsers as opposed to scripts. For example, Mozilla Firefox may identify itself as `"Mozilla/5.0 (X11; U; Linux i686) Gecko/20071127 Firefox/2.0.0.11"`, while `urllib`'s default user agent string is `"Python-urllib/2.6"` (on Python 2.6). All header keys are

sent in camel case.

An appropriate `Content-Type` header should be included if the *data* argument is present. If this header has not been provided and *data* is not `None`, `Content-Type: application/x-www-form-urlencoded` will be added as a default.

The next two arguments are only of interest for correct handling of third-party HTTP cookies:

origin_req_host should be the request-host of the origin transaction, as defined by **RFC 2965**. It defaults to `http.cookiejar.request_host(self)`. This is the host name or IP address of the original request that was initiated by the user. For example, if the request is for an image in an HTML document, this should be the request-host of the request for the page containing the image.

unverifiable should indicate whether the request is unverifiable, as defined by **RFC 2965**. It defaults to `False`. An unverifiable request is one whose URL the user did not have the option to approve. For example, if the request is for an image in an HTML document, and the user had no option to approve the automatic fetching of the image, this should be true.

method should be a string that indicates the HTTP request method that will be used (e.g. `'HEAD'`). If provided, its value is stored in the `method` attribute and is used by `get_method()`. The default is `'GET'` if *data* is `None` or `'POST'` otherwise. Subclasses may indicate a different default method by setting the `method` attribute in the class itself.

> **Note:** The request will not work as expected if the data object is unable to deliver its content more than once (e.g. a file or an iterable that can produce the content only once) and the request is retried for HTTP redirects or authentication. The *data* is sent to the HTTP server right away after the headers. There is no support for a 100-continue expectation in the library.

> *Changed in version 3.3:* `Request.method` argument is added to the Request class.

> *Changed in version 3.4:* Default `Request.method` may be indicated at the class level.

> *Changed in version 3.6:* Do not raise an error if the `Content-Length` has not been provided and *data* is neither `None` nor a bytes object. Fall back to use chunked transfer encoding instead.

class `urllib.request.`**`OpenerDirector`**

The `OpenerDirector` class opens URLs via `BaseHandler`s chained together. It manages the chaining of handlers, and recovery from errors.

class `urllib.request.`**`BaseHandler`**

This is the base class for all registered handlers — and handles only the simple mechanics of registration.

class `urllib.request.`**`HTTPDefaultErrorHandler`**

A class which defines a default handler for HTTP error responses; all responses are turned into `HTTPError` exceptions.

class urllib.request.**HTTPRedirectHandler**

A class to handle redirections.

class urllib.request.**HTTPCookieProcessor**(*cookiejar=None*)

A class to handle HTTP Cookies.

class urllib.request.**ProxyHandler**(*proxies=None*)

Cause requests to go through a proxy. If *proxies* is given, it must be a dictionary mapping protocol names to URLs of proxies. The default is to read the list of proxies from the environment variables `<protocol>_proxy`. If no proxy environment variables are set, then in a Windows environment proxy settings are obtained from the registry's Internet Settings section, and in a macOS environment proxy information is retrieved from the System Configuration Framework.

To disable autodetected proxy pass an empty dictionary.

The `no_proxy` environment variable can be used to specify hosts which shouldn't be reached via proxy; if set, it should be a comma-separated list of hostname suffixes, optionally with `:port` appended, for example `cern.ch,ncsa.uiuc.edu,some.host:8080`.

> **Note:** HTTP_PROXY will be ignored if a variable REQUEST_METHOD is set; see the documentation on `getproxies()`.

class urllib.request.**HTTPPasswordMgr**

Keep a database of `(realm, uri) -> (user, password)` mappings.

class urllib.request.**HTTPPasswordMgrWithDefaultRealm**

Keep a database of `(realm, uri) -> (user, password)` mappings. A realm of None is considered a catch-all realm, which is searched if no other realm fits.

class urllib.request.**HTTPPasswordMgrWithPriorAuth**

A variant of `HTTPPasswordMgrWithDefaultRealm` that also has a database of `uri -> is_authenticated` mappings. Can be used by a BasicAuth handler to determine when to send authentication credentials immediately instead of waiting for a `401` response first.

> *Added in version 3.5.*

class urllib.request.**AbstractBasicAuthHandler**(*password_mgr=None*)

This is a mixin class that helps with HTTP authentication, both to the remote host and to a proxy. *password_mgr*, if given, should be something that is compatible with `HTTPPasswordMgr`; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported. If *passwd_mgr* also provides `is_authenticated` and `update_authenticated` methods (see HTTPPasswordMgrWithPriorAuth Objects), then the handler will use the `is_authenticated` result for a given URI to determine whether or not to send authentication credentials with the request. If `is_authenticated` returns `True` for the URI, credentials are sent. If `is_authenticated` is `False`, credentials are not sent, and then if a `401` response is received the request is re-sent with the authentication credentials. If authentication succeeds, `update_authenticated` is called to set `is_authenticated` `True` for the URI, so that subsequent requests to the URI or any of its super-URIs will automatically include the authentication

credentials.

> *Added in version 3.5:* Added `is_authenticated` support.

class urllib.request.**HTTPBasicAuthHandler**(*password_mgr=None*)

Handle authentication with the remote host. *password_mgr*, if given, should be something that is compatible with HTTPPasswordMgr; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported. HTTPBasicAuthHandler will raise a ValueError when presented with a wrong Authentication scheme.

class urllib.request.**ProxyBasicAuthHandler**(*password_mgr=None*)

Handle authentication with the proxy. *password_mgr*, if given, should be something that is compatible with HTTPPasswordMgr; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported.

class urllib.request.**AbstractDigestAuthHandler**(*password_mgr=None*)

This is a mixin class that helps with HTTP authentication, both to the remote host and to a proxy. *password_mgr*, if given, should be something that is compatible with HTTPPasswordMgr; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported.

> *Changed in version 3.14:* Added support for HTTP digest authentication algorithm `SHA-256`.

class urllib.request.**HTTPDigestAuthHandler**(*password_mgr=None*)

Handle authentication with the remote host. *password_mgr*, if given, should be something that is compatible with HTTPPasswordMgr; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported. When both Digest Authentication Handler and Basic Authentication Handler are both added, Digest Authentication is always tried first. If the Digest Authentication returns a 40x response again, it is sent to Basic Authentication handler to Handle. This Handler method will raise a ValueError when presented with an authentication scheme other than Digest or Basic.

> *Changed in version 3.3:* Raise ValueError on unsupported Authentication Scheme.

class urllib.request.**ProxyDigestAuthHandler**(*password_mgr=None*)

Handle authentication with the proxy. *password_mgr*, if given, should be something that is compatible with HTTPPasswordMgr; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported.

class urllib.request.**HTTPHandler**

A class to handle opening of HTTP URLs.

class urllib.request.**HTTPSHandler**(*debuglevel=0, context=None, check_hostname=None*)

A class to handle opening of HTTPS URLs. *context* and *check_hostname* have the same meaning as in http.client.HTTPSConnection.

> *Changed in version 3.2:* *context* and *check_hostname* were added.

class urllib.request.**FileHandler**

Open local files.

class urllib.request.**DataHandler**

Open data URLs.

> *Added in version 3.4.*

class urllib.request.**FTPHandler**

Open FTP URLs.

class urllib.request.**CacheFTPHandler**

Open FTP URLs, keeping a cache of open FTP connections to minimize delays.

class urllib.request.**UnknownHandler**

A catch-all class to handle unknown URLs.

class urllib.request.**HTTPErrorProcessor**

Process HTTP error responses.

Request Objects

The following methods describe `Request`'s public interface, and so all may be overridden in subclasses. It also defines several public attributes that can be used by clients to inspect the parsed request.

Request.**full_url**

The original URL passed to the constructor.

> *Changed in version 3.4.*

Request.full_url is a property with setter, getter and a deleter. Getting `full_url` returns the original request URL with the fragment, if it was present.

Request.**type**

The URI scheme.

Request.**host**

The URI authority, typically a host, but may also contain a port separated by a colon.

Request.**origin_req_host**

The original host for the request, without port.

Request.**selector**

The URI path. If the `Request` uses a proxy, then selector will be the full URL that is passed to the proxy.

Request.**data**

The entity body for the request, or None if not specified.

> *Changed in version 3.4:* Changing value of `Request.data` now deletes "Content-Length" header if it was previously set or calculated.

Request.**unverifiable**

boolean, indicates whether the request is unverifiable as defined by **RFC 2965**.

Request.**method**

The HTTP request method to use. By default its value is None, which means that get_method() will do its normal computation of the method to be used. Its value can be set (thus overriding the default computation in get_method()) either by providing a default value by setting it at the class level in a Request subclass, or by passing a value in to the Request constructor via the *method* argument.

> *Added in version 3.3.*

> *Changed in version 3.4:* A default value can now be set in subclasses; previously it could only be set via the constructor argument.

Request.**get_method**()

Return a string indicating the HTTP request method. If Request.method is not None, return its value, otherwise return 'GET' if Request.data is None, or 'POST' if it's not. This is only meaningful for HTTP requests.

> *Changed in version 3.3:* get_method now looks at the value of Request.method.

Request.**add_header**(*key, val*)

Add another header to the request. Headers are currently ignored by all handlers except HTTP handlers, where they are added to the list of headers sent to the server. Note that there cannot be more than one header with the same name, and later calls will overwrite previous calls in case the *key* collides. Currently, this is no loss of HTTP functionality, since all headers which have meaning when used more than once have a (header-specific) way of gaining the same functionality using only one header. Note that headers added using this method are also added to redirected requests.

Request.**add_unredirected_header**(*key, header*)

Add a header that will not be added to a redirected request.

Request.**has_header**(*header*)

Return whether the instance has the named header (checks both regular and unredirected).

Request.**remove_header**(*header*)

Remove named header from the request instance (both from regular and unredirected headers).

> *Added in version 3.4.*

Request.**get_full_url**()

Return the URL given in the constructor.

> *Changed in version 3.4.*

Returns Request.full_url

Request.**set_proxy**(*host, type*)

Prepare the request by connecting to a proxy server. The *host* and *type* will replace those of the instance, and the instance's selector will be the original URL given in the constructor.

`Request.`**`get_header`**`(`*`header_name, default=None`*`)`

Return the value of the given header. If the header is not present, return the default value.

`Request.`**`header_items`**`()`

Return a list of tuples (header_name, header_value) of the Request headers.

> *Changed in version 3.4:* The request methods add_data, has_data, get_data, get_type, get_host, get_selector, get_origin_req_host and is_unverifiable that were deprecated since 3.3 have been removed.

OpenerDirector Objects

`OpenerDirector` instances have the following methods:

`OpenerDirector.`**`add_handler`**`(`*`handler`*`)`

handler should be an instance of `BaseHandler`. The following methods are searched, and added to the possible chains (note that HTTP errors are a special case). Note that, in the following, *protocol* should be replaced with the actual protocol to handle, for example `http_response()` would be the HTTP protocol response handler. Also *type* should be replaced with the actual HTTP code, for example `http_error_404()` would handle HTTP 404 errors.

- `<protocol>_open()` — signal that the handler knows how to open *protocol* URLs.

  See `BaseHandler.<protocol>_open()` for more information.

- `http_error_<type>()` — signal that the handler knows how to handle HTTP errors with HTTP error code *type*.

  See `BaseHandler.http_error_<nnn>()` for more information.

- `<protocol>_error()` — signal that the handler knows how to handle errors from (non-`http`) *protocol*.

- `<protocol>_request()` — signal that the handler knows how to pre-process *protocol* requests.

  See `BaseHandler.<protocol>_request()` for more information.

- `<protocol>_response()` — signal that the handler knows how to post-process *protocol* responses.

  See `BaseHandler.<protocol>_response()` for more information.

`OpenerDirector.`**`open`**`(`*`url, data=None`*`[, `*`timeout`*`])`

Open the given *url* (which can be a request object or a string), optionally passing the given *data*. Arguments, return values and exceptions raised are the same as those of `urlopen()` (which simply calls the `open()` method on the currently installed global `OpenerDirector`). The optional

timeout parameter specifies a timeout in seconds for blocking operations like the connection attempt (if not specified, the global default timeout setting will be used). The timeout feature actually works only for HTTP, HTTPS and FTP connections.

`OpenerDirector.`**`error`**`(`*`proto`*`, *`*`args`*`)`

Handle an error of the given protocol. This will call the registered error handlers for the given protocol with the given arguments (which are protocol specific). The HTTP protocol is a special case which uses the HTTP response code to determine the specific error handler; refer to the `http_error_<type>()` methods of the handler classes.

Return values and exceptions raised are the same as those of `urlopen()`.

OpenerDirector objects open URLs in three stages:

The order in which these methods are called within each stage is determined by sorting the handler instances.

1. Every handler with a method named like `<protocol>_request()` has that method called to preprocess the request.

2. Handlers with a method named like `<protocol>_open()` are called to handle the request. This stage ends when a handler either returns a non-`None` value (ie. a response), or raises an exception (usually `URLError`). Exceptions are allowed to propagate.

   In fact, the above algorithm is first tried for methods named `default_open()`. If all such methods return `None`, the algorithm is repeated for methods named like `<protocol>_open()`. If all such methods return `None`, the algorithm is repeated for methods named `unknown_open()`.

   Note that the implementation of these methods may involve calls of the parent `OpenerDirector` instance's `open()` and `error()` methods.

3. Every handler with a method named like `<protocol>_response()` has that method called to post-process the response.

BaseHandler Objects

`BaseHandler` objects provide a couple of methods that are directly useful, and others that are meant to be used by derived classes. These are intended for direct use:

`BaseHandler.`**`add_parent`**`(`*`director`*`)`

Add a director as parent.

`BaseHandler.`**`close`**`()`

Remove any parents.

The following attribute and methods should only be used by classes derived from `BaseHandler`.

> **Note:** The convention has been adopted that subclasses defining `<protocol>_request()` or `<protocol>_response()` methods are named `*Processor`; all others are named `*Handler`.

`BaseHandler.`**`parent`**

A valid `OpenerDirector`, which can be used to open using a different protocol, or handle errors.

`BaseHandler.`**`default_open`**`(`*`req`*`)`

This method is *not* defined in `BaseHandler`, but subclasses should define it if they want to catch all URLs.

This method, if implemented, will be called by the parent `OpenerDirector`. It should return a file-like object as described in the return value of the `open()` method of `OpenerDirector`, or `None`. It should raise `URLError`, unless a truly exceptional thing happens (for example, `MemoryError` should not be mapped to `URLError`).

This method will be called before any protocol-specific open method.

`BaseHandler.<protocol>_open(req)`

This method is *not* defined in `BaseHandler`, but subclasses should define it if they want to handle URLs with the given protocol.

This method, if defined, will be called by the parent `OpenerDirector`. Return values should be the same as for `default_open()`.

`BaseHandler.`**`unknown_open`**`(`*`req`*`)`

This method is *not* defined in `BaseHandler`, but subclasses should define it if they want to catch all URLs with no specific registered handler to open it.

This method, if implemented, will be called by the `parent` `OpenerDirector`. Return values should be the same as for `default_open()`.

`BaseHandler.`**`http_error_default`**`(`*`req, fp, code, msg, hdrs`*`)`

This method is *not* defined in `BaseHandler`, but subclasses should override it if they intend to provide a catch-all for otherwise unhandled HTTP errors. It will be called automatically by the `OpenerDirector` getting the error, and should not normally be called in other circumstances.

`OpenerDirector` will call this method with five positional arguments:

1. a `Request` object,
2. a file-like object with the HTTP error body,
3. the three-digit code of the error, as a string,
4. the user-visible explanation of the code, as a string, and
5. the headers of the error, as a mapping object.

Return values and exceptions raised should be the same as those of `urlopen()`.

`BaseHandler.http_error_<nnn>(req, fp, code, msg, hdrs)`

nnn should be a three-digit HTTP error code. This method is also not defined in `BaseHandler`, but will be called, if it exists, on an instance of a subclass, when an HTTP error with code *nnn* occurs.

Subclasses should override this method to handle specific HTTP errors.

Arguments, return values and exceptions raised should be the same as for `http_error_default()`.

`BaseHandler.<protocol>_request(req)`

This method is *not* defined in `BaseHandler`, but subclasses should define it if they want to pre-process requests of the given protocol.

This method, if defined, will be called by the parent `OpenerDirector`. *req* will be a `Request` object. The return value should be a `Request` object.

`BaseHandler.<protocol>_response(req, response)`

This method is *not* defined in `BaseHandler`, but subclasses should define it if they want to post-process responses of the given protocol.

This method, if defined, will be called by the parent `OpenerDirector`. *req* will be a `Request` object. *response* will be an object implementing the same interface as the return value of `urlopen()`. The return value should implement the same interface as the return value of `urlopen()`.

HTTPRedirectHandler Objects

Note: Some HTTP redirections require action from this module's client code. If this is the case, `HTTPError` is raised. See **RFC 2616** for details of the precise meanings of the various redirection codes.

An `HTTPError` exception raised as a security consideration if the HTTPRedirectHandler is presented with a redirected URL which is not an HTTP, HTTPS or FTP URL.

`HTTPRedirectHandler.`**`redirect_request`**`(`*`req`*`, `*`fp`*`, `*`code`*`, `*`msg`*`, `*`hdrs`*`, `*`newurl`*`)`

Return a `Request` or `None` in response to a redirect. This is called by the default implementations of the `http_error_30*()` methods when a redirection is received from the server. If a redirection should take place, return a new `Request` to allow `http_error_30*()` to perform the redirect to *newurl*. Otherwise, raise `HTTPError` if no other handler should try to handle this URL, or return `None` if you can't but another handler might.

Note: The default implementation of this method does not strictly follow **RFC 2616**, which says that 301 and 302 responses to `POST` requests must not be automatically redirected without confirmation by the user. In reality, browsers do allow automatic redirection of these responses, changing the POST to a `GET`, and the default implementation reproduces this behavior.

`HTTPRedirectHandler.`**`http_error_301`**`(`*`req`*`, `*`fp`*`, `*`code`*`, `*`msg`*`, `*`hdrs`*`)`

Redirect to the `Location:` or `URI:` URL. This method is called by the parent `OpenerDirector` when getting an HTTP 'moved permanently' response.

`HTTPRedirectHandler.`**`http_error_302`**`(`*`req`*`, `*`fp`*`, `*`code`*`, `*`msg`*`, `*`hdrs`*`)`

The same as `http_error_301()`, but called for the 'found' response.

HTTPRedirectHandler.**http_error_303**(*req*, *fp*, *code*, *msg*, *hdrs*)

The same as http_error_301(), but called for the 'see other' response.

HTTPRedirectHandler.**http_error_307**(*req*, *fp*, *code*, *msg*, *hdrs*)

The same as http_error_301(), but called for the 'temporary redirect' response. It does not allow changing the request method from POST to GET.

HTTPRedirectHandler.**http_error_308**(*req*, *fp*, *code*, *msg*, *hdrs*)

The same as http_error_301(), but called for the 'permanent redirect' response. It does not allow changing the request method from POST to GET.

> *Added in version 3.11.*

HTTPCookieProcessor Objects

HTTPCookieProcessor instances have one attribute:

HTTPCookieProcessor.**cookiejar**

The http.cookiejar.CookieJar in which cookies are stored.

ProxyHandler Objects

ProxyHandler.<protocol>_open(request)

The ProxyHandler will have a method <protocol>_open() for every *protocol* which has a proxy in the *proxies* dictionary given in the constructor. The method will modify requests to go through the proxy, by calling request.set_proxy(), and call the next handler in the chain to actually execute the protocol.

HTTPPasswordMgr Objects

These methods are available on HTTPPasswordMgr and HTTPPasswordMgrWithDefaultRealm objects.

HTTPPasswordMgr.**add_password**(*realm*, *uri*, *user*, *passwd*)

uri can be either a single URI, or a sequence of URIs. *realm*, *user* and *passwd* must be strings. This causes (user, passwd) to be used as authentication tokens when authentication for *realm* and a super-URI of any of the given URIs is given.

HTTPPasswordMgr.**find_user_password**(*realm*, *authuri*)

Get user/password for given realm and URI, if any. This method will return (None, None) if there is no matching user/password.

For HTTPPasswordMgrWithDefaultRealm objects, the realm None will be searched if the given *realm* has no matching user/password.

HTTPPasswordMgrWithPriorAuth Objects

This password manager extends HTTPPasswordMgrWithDefaultRealm to support tracking URIs for

which authentication credentials should always be sent.

`HTTPPasswordMgrWithPriorAuth.`**`add_password`**`(`*`realm`*`, `*`uri`*`, `*`user`*`, `*`passwd`*`, `*`is_authenticated=False`*`)`

realm, *uri*, *user*, *passwd* are as for `HTTPPasswordMgr.add_password()`. *is_authenticated* sets the initial value of the `is_authenticated` flag for the given URI or list of URIs. If *is_authenticated* is specified as `True`, *realm* is ignored.

`HTTPPasswordMgrWithPriorAuth.`**`find_user_password`**`(`*`realm`*`, `*`authuri`*`)`

Same as for `HTTPPasswordMgrWithDefaultRealm` objects

`HTTPPasswordMgrWithPriorAuth.`**`update_authenticated`**`(`*`self`*`, `*`uri`*`, `*`is_authenticated=False`*`)`

Update the `is_authenticated` flag for the given *uri* or list of URIs.

`HTTPPasswordMgrWithPriorAuth.`**`is_authenticated`**`(`*`self`*`, `*`authuri`*`)`

Returns the current state of the `is_authenticated` flag for the given URI.

AbstractBasicAuthHandler Objects

`AbstractBasicAuthHandler.`**`http_error_auth_reqed`**`(`*`authreq`*`, `*`host`*`, `*`req`*`, `*`headers`*`)`

Handle an authentication request by getting a user/password pair, and re-trying the request. *authreq* should be the name of the header where the information about the realm is included in the request, *host* specifies the URL and path to authenticate for, *req* should be the (failed) `Request` object, and *headers* should be the error headers.

host is either an authority (e.g. `"python.org"`) or a URL containing an authority component (e.g. `"https://python.org/"`). In either case, the authority must not contain a userinfo component (so, `"python.org"` and `"python.org:80"` are fine, `"joe:password@python.org"` is not).

HTTPBasicAuthHandler Objects

`HTTPBasicAuthHandler.`**`http_error_401`**`(`*`req`*`, `*`fp`*`, `*`code`*`, `*`msg`*`, `*`hdrs`*`)`

Retry the request with authentication information, if available.

ProxyBasicAuthHandler Objects

`ProxyBasicAuthHandler.`**`http_error_407`**`(`*`req`*`, `*`fp`*`, `*`code`*`, `*`msg`*`, `*`hdrs`*`)`

Retry the request with authentication information, if available.

AbstractDigestAuthHandler Objects

`AbstractDigestAuthHandler.`**`http_error_auth_reqed`**`(`*`authreq`*`, `*`host`*`, `*`req`*`, `*`headers`*`)`

authreq should be the name of the header where the information about the realm is included in the request, *host* should be the host to authenticate to, *req* should be the (failed) `Request` object,

and *headers* should be the error headers.

HTTPDigestAuthHandler Objects

`HTTPDigestAuthHandler.`**`http_error_401`**`(`*`req`*`, `*`fp`*`, `*`code`*`, `*`msg`*`, `*`hdrs`*`)`

Retry the request with authentication information, if available.

ProxyDigestAuthHandler Objects

`ProxyDigestAuthHandler.`**`http_error_407`**`(`*`req`*`, `*`fp`*`, `*`code`*`, `*`msg`*`, `*`hdrs`*`)`

Retry the request with authentication information, if available.

HTTPHandler Objects

`HTTPHandler.`**`http_open`**`(`*`req`*`)`

Send an HTTP request, which can be either GET or POST, depending on `req.data`.

HTTPSHandler Objects

`HTTPSHandler.`**`https_open`**`(`*`req`*`)`

Send an HTTPS request, which can be either GET or POST, depending on `req.data`.

FileHandler Objects

`FileHandler.`**`file_open`**`(`*`req`*`)`

Open the file locally, if there is no host name, or the host name is `'localhost'`.

> *Changed in version 3.2:* This method is applicable only for local hostnames. When a remote hostname is given, a `URLError` is raised.

DataHandler Objects

`DataHandler.`**`data_open`**`(`*`req`*`)`

Read a data URL. This kind of URL contains the content encoded in the URL itself. The data URL syntax is specified in **RFC 2397**. This implementation ignores white spaces in base64 encoded data URLs so the URL may be wrapped in whatever source file it comes from. But even though some browsers don't mind about a missing padding at the end of a base64 encoded data URL, this implementation will raise a `ValueError` in that case.

FTPHandler Objects

`FTPHandler.`**`ftp_open`**`(`*`req`*`)`

Open the FTP file indicated by *req*. The login is always done with empty username and password.

CacheFTPHandler Objects

`CacheFTPHandler` objects are `FTPHandler` objects with the following additional methods:

`CacheFTPHandler.`**`setTimeout`**`(`*`t`*`)`

Set timeout of connections to *t* seconds.

`CacheFTPHandler.`**`setMaxConns`**`(`*`m`*`)`

Set maximum number of cached connections to *m*.

UnknownHandler Objects

`UnknownHandler.`**`unknown_open`**`()`

Raise a `URLError` exception.

HTTPErrorProcessor Objects

`HTTPErrorProcessor.`**`http_response`**`(`*`request, response`*`)`

Process HTTP error responses.

For 200 error codes, the response object is returned immediately.

For non-200 error codes, this simply passes the job on to the `http_error_<type>()` handler methods, via `OpenerDirector.error()`. Eventually, `HTTPDefaultErrorHandler` will raise an `HTTPError` if no other handler handles the error.

`HTTPErrorProcessor.`**`https_response`**`(`*`request, response`*`)`

Process HTTPS error responses.

The behavior is same as `http_response()`.

Examples

In addition to the examples below, more examples are given in HOWTO Fetch Internet Resources Using The urllib Package.

This example gets the python.org main page and displays the first 300 bytes of it:

```
>>> import urllib.request
>>> with urllib.request.urlopen('https://www.python.org/') as f:
...     # The response may be compressed (for example, 'gzip').
...     print(f.headers.get('Content-Encoding'))
...     data = f.read()
...     if f.headers.get('Content-Encoding') == 'gzip':
...         import gzip
...         data = gzip.decompress(data)
...     print(data[:300].decode('utf-8', errors='replace'))
```

Note that urlopen returns a bytes object. This is because there is no way for urlopen to automatically determine the encoding of the byte stream it receives from the HTTP server. In general, a program will decode the returned bytes object to string once it determines or guesses the appropriate encoding.

The following HTML spec document, https://html.spec.whatwg.org/#charset, lists the various ways in

which an HTML or an XML document could have specified its encoding information.

For additional information, see the W3C document: https://www.w3.org/International/questions/qa-html-encoding-declarations.

As the python.org website uses *utf-8* encoding as specified in its meta tag, we will use the same for decoding the bytes object:

```
>>> with urllib.request.urlopen('https://www.python.org/') as f:
...     # Check for compression and decode appropriately.
...     enc = f.headers.get('Content-Encoding')
...     data = f.read()
...     if enc == 'gzip':
...         import gzip
...         data = gzip.decompress(data)
...     print(data[:100].decode('utf-8', errors='replace'))
...
```

It is also possible to achieve the same result without using the context manager approach:

```
>>> import urllib.request
>>> f = urllib.request.urlopen('https://www.python.org/')
>>> try:
...     enc = f.headers.get('Content-Encoding')
...     data = f.read()
...     if enc == 'gzip':
...         import gzip
...         data = gzip.decompress(data)
...     print(data[:100].decode('utf-8', errors='replace'))
... finally:
...     f.close()
```

In the following example, we are sending a data-stream to the stdin of a CGI and reading the data it returns to us. Note that this example will only work when the Python installation supports SSL.

```
>>> import urllib.request
>>> req = urllib.request.Request(url='https://localhost/cgi-bin/test.cgi',
...                       data=b'This data is passed to stdin of the CGI')
>>> with urllib.request.urlopen(req) as f:
...     print(f.read().decode('utf-8'))
...
Got Data: "This data is passed to stdin of the CGI"
```

The code for the sample CGI used in the above example is:

```
#!/usr/bin/env python
import sys
data = sys.stdin.read()
print('Content-type: text/plain\n\nGot Data: "%s"' % data)
```

Here is an example of doing a PUT request using `Request`:

```
import urllib.request
DATA = b'some data'
req = urllib.request.Request(url='http://localhost:8080', data=DATA, method='PUT'
with urllib.request.urlopen(req) as f:
    pass
```

```
print(f.status)
print(f.reason)
```

Use of Basic HTTP Authentication:

```
import urllib.request
# Create an OpenerDirector with support for Basic HTTP Authentication...
auth_handler = urllib.request.HTTPBasicAuthHandler()
auth_handler.add_password(realm='PDQ Application',
                          uri='https://mahler:8092/site-updates.py',
                          user='klem',
                          passwd='kadidd!ehopper')
opener = urllib.request.build_opener(auth_handler)
# ...and install it globally so it can be used with urlopen.
urllib.request.install_opener(opener)
with urllib.request.urlopen('http://www.example.com/login.html') as f:
    print(f.read().decode('utf-8'))
```

`build_opener()` provides many handlers by default, including a `ProxyHandler`. By default, `ProxyHandler` uses the environment variables named <scheme>_proxy, where <scheme> is the URL scheme involved. For example, the `http_proxy` environment variable is read to obtain the HTTP proxy's URL.

This example replaces the default `ProxyHandler` with one that uses programmatically supplied proxy URLs, and adds proxy authorization support with `ProxyBasicAuthHandler`.

```
proxy_handler = urllib.request.ProxyHandler({'http': 'http://www.example.com:3128/
proxy_auth_handler = urllib.request.ProxyBasicAuthHandler()
proxy_auth_handler.add_password('realm', 'host', 'username', 'password')

opener = urllib.request.build_opener(proxy_handler, proxy_auth_handler)
# This time, rather than install the OpenerDirector, we use it directly:
with opener.open('http://www.example.com/login.html') as f:
   print(f.read().decode('utf-8'))
```

Adding HTTP headers:

Use the *headers* argument to the `Request` constructor, or:

```
import urllib.request
req = urllib.request.Request('http://www.example.com/')
req.add_header('Referer', 'https://www.python.org/')
# Customize the default User-Agent header value:
req.add_header('User-Agent', 'urllib-example/0.1 (Contact: . . .)')
with urllib.request.urlopen(req) as f:
    print(f.read().decode('utf-8'))
```

`OpenerDirector` automatically adds a *User-Agent* header to every `Request`. To change this:

```
import urllib.request
opener = urllib.request.build_opener()
opener.addheaders = [('User-agent', 'Mozilla/5.0')]
with opener.open('http://www.example.com/') as f:
   print(f.read().decode('utf-8'))
```

Also, remember that a few standard headers (*Content-Length*, *Content-Type* and *Host*) are added

when the Request is passed to urlopen() (or OpenerDirector.open()).

Here is an example session that uses the GET method to retrieve a URL containing parameters:

```
>>> import urllib.request
>>> import urllib.parse
>>> params = urllib.parse.urlencode({'spam': 1, 'eggs': 2, 'bacon': 0})
>>> url = "https://www.python.org/?%s" % params
>>> with urllib.request.urlopen(url) as f:
...     print(f.read().decode('utf-8'))
...
```

The following example uses the POST method instead. Note that params output from urlencode is encoded to bytes before it is sent to urlopen as data:

```
>>> import urllib.request
>>> import urllib.parse
>>> data = urllib.parse.urlencode({'spam': 1, 'eggs': 2, 'bacon': 0})
>>> data = data.encode('ascii')
>>> with urllib.request.urlopen("https://httpbin.org/post", data) as f:
...     print(f.read().decode('utf-8'))
...
```

The following example uses an explicitly specified HTTP proxy, overriding environment settings:

```
>>> import urllib.request
>>> proxies = {'http': 'http://proxy.example.com:8080/'}
>>> opener = urllib.request.build_opener(urllib.request.ProxyHandler(proxies))
>>> with opener.open("https://www.python.org") as f:
...     f.read().decode('utf-8')
...
```

The following example uses no proxies at all, overriding environment settings:

```
>>> import urllib.request
>>> opener = urllib.request.build_opener(urllib.request.ProxyHandler({}))
>>> with opener.open("https://www.python.org/") as f:
...     f.read().decode('utf-8')
...
```

Legacy interface

The following functions and classes are ported from the Python 2 module `urllib` (as opposed to `urllib2`). They might become deprecated at some point in the future.

`urllib.request.`**`urlretrieve`**`(`*`url, filename=None, reporthook=None, data=None`*`)`

Copy a network object denoted by a URL to a local file. If the URL points to a local file, the object will not be copied unless filename is supplied. Return a tuple `(filename, headers)` where *filename* is the local file name under which the object can be found, and *headers* is whatever the `info()` method of the object returned by urlopen() returned (for a remote object). Exceptions are the same as for `urlopen()`.

The second argument, if present, specifies the file location to copy to (if absent, the location will be a tempfile with a generated name). The third argument, if present, is a callable that will be

called once on establishment of the network connection and once after each block read thereafter. The callable will be passed three arguments; a count of blocks transferred so far, a block size in bytes, and the total size of the file. The third argument may be `-1` on older FTP servers which do not return a file size in response to a retrieval request.

The following example illustrates the most common usage scenario:

```
>>> import urllib.request
>>> local_filename, headers = urllib.request.urlretrieve('https://python.org/'
>>> html = open(local_filename)
>>> html.close()
```

If the *url* uses the `http:` scheme identifier, the optional *data* argument may be given to specify a `POST` request (normally the request type is `GET`). The *data* argument must be a bytes object in standard *application/x-www-form-urlencoded* format; see the `urllib.parse.urlencode()` function.

`urlretrieve()` will raise `ContentTooShortError` when it detects that the amount of data available was less than the expected amount (which is the size reported by a *Content-Length* header). This can occur, for example, when the download is interrupted.

The *Content-Length* is treated as a lower bound: if there's more data to read, urlretrieve reads more data, but if less data is available, it raises the exception.

You can still retrieve the downloaded data in this case, it is stored in the `content` attribute of the exception instance.

If no *Content-Length* header was supplied, urlretrieve can not check the size of the data it has downloaded, and just returns it. In this case you just have to assume that the download was successful.

`urllib.request.`**`urlcleanup`**`()`

Cleans up temporary files that may have been left behind by previous calls to `urlretrieve()`.

`urllib.request` Restrictions

- Currently, only the following protocols are supported: HTTP (versions 0.9 and 1.0), FTP, local files, and data URLs.

  > *Changed in version 3.4:* Added support for data URLs.

- The caching feature of `urlretrieve()` has been disabled until someone finds the time to hack proper processing of Expiration time headers.

- There should be a function to query whether a particular URL is in the cache.

- For backward compatibility, if a URL appears to point to a local file but the file can't be opened, the URL is re-interpreted using the FTP protocol. This can sometimes cause confusing error messages.

- The `urlopen()` and `urlretrieve()` functions can cause arbitrarily long delays while waiting for a network connection to be set up. This means that it is difficult to build an interactive web client us-

ing these functions without using threads.

- The data returned by `urlopen()` or `urlretrieve()` is the raw data returned by the server. This may be binary data (such as an image), plain text or (for example) HTML. The HTTP protocol provides type information in the reply header, which can be inspected by looking at the *Content-Type* header. If the returned data is HTML, you can use the module `html.parser` to parse it.

- The code handling the FTP protocol cannot differentiate between a file and a directory. This can lead to unexpected behavior when attempting to read a URL that points to a file that is not accessible. If the URL ends in a `/`, it is assumed to refer to a directory and will be handled accordingly. But if an attempt to read a file leads to a 550 error (meaning the URL cannot be found or is not accessible, often for permission reasons), then the path is treated as a directory in order to handle the case when a directory is specified by a URL but the trailing `/` has been left off. This can cause misleading results when you try to fetch a file whose read permissions make it inaccessible; the FTP code will try to read it, fail with a 550 error, and then perform a directory listing for the unreadable file. If fine-grained control is needed, consider using the `ftplib` module.

`urllib.response` — Response classes used by urllib

The `urllib.response` module defines functions and classes which define a minimal file-like interface, including `read()` and `readline()`. Functions defined by this module are used internally by the `urllib.request` module. The typical response object is a `urllib.response.addinfourl` instance:

class urllib.response.**addinfourl**

url

URL of the resource retrieved, commonly used to determine if a redirect was followed.

headers

Returns the headers of the response in the form of an `EmailMessage` instance.

status

> *Added in version 3.9.*

Status code returned by server.

geturl()

> *Deprecated since version 3.9:* Deprecated in favor of `url`.

info()

> *Deprecated since version 3.9:* Deprecated in favor of `headers`.

code

> *Deprecated since version 3.9:* Deprecated in favor of `status`.

getcode()

> *Deprecated since version 3.9:* Deprecated in favor of `status`.